Aalto University
School of Electrical
Engineering

Master's Programme in Computer, Communication and Information Sciences

MPC in practice

MPC in practice

Tianxing Wu

Master's Thesis
2024

© 2024

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International" license.

Aalto University
School of Electrical
Engineering

Author Tianxing Wu

Title MPC in practice — MPC in practice

Degree programme Computer, Communication and Information Sciences

Major Algorithms

Supervisor Prof. Jukka Suomela

Advisors Dr Alan Advisor, Hossein Vahidi

Date 10 July 2024 **Number of pages** 13 **Language** English

Abstract

Purpose:
The purpose of this thesis is to study how congested clique algorithms perform on modern supercomputers.
Methodology:
Studied how an MST algorithm performs on the Puhti supercomputer. The congested clique setting is simulated with MPI_and CUDA.
Result:

Keywords Distributed computing, Supercomputing, Congested clique, MST, Puhti, MPI, CUDA, MPC

Preface

I want to thank Professor Jukka Suomela and my advisor Hossein Vahidi for their guidance.

I also want to thank my friends for keeping me sane and alive.

Otaniemi, 10 July 2024

Tianxing Wu

Contents

Abstract **3**

Preface **4**

Contents **5**

Symbols and abbreviations **6**

1 Introduction **7**
1.1 The MST problem 8
1.2 The algorithm A 8
1.3 The super-computer Puhti 8

2 CPU implementation **9**
2.1 Congested Clique simulation 9
2.2 Memory limit and graph generation 9

3 CPU Algorithm performance **10**
3.1 A time composition 10
3.2 performance compared to sequential algorithm 10
3.3 Scalability 10
3.4 Python vs C++ 10

4 Cuda implementation **12**

References **13**

Symbols and abbreviations

Abbreviations

A The algorithm described in [1]

1 Introduction

Congested Clique, described in [2] is a mathematical model for distributed computing. It features n inter-connected players in a clique network, sending messages in a synchronized manner. In each round, each player sends $O(\log n)$ to each other player. The computation efficiency is measured by the communication rounds. In this work, the congested clique algorithm that solves the MST in $O(\log \log n)$ rounds [1] is studied and implemented on the supercomputer Puhti [3]. We simulated the congested clique model with CPU threads and a CUDA GPU to run the MST algorithm. The purpose of this work is to identify the connection between A and modern distributed computing hardware and tools. The relations allow us to understand the bigger picture as A is representative of the congested clique model

The representativeness of A can be shown in the following ways:

- A exploit the full connectivity of the graph. In each round, there is a step where each player sends $O(\log n)$ messages to all the other players.
- To comply with the $O(\log n)$ limit when sending to a unique leader, large data is spilled to other players before sending.
- The algorithm needs several rounds before its convergence.

We measure how compatible A is with the Supercomputing infrastructure to find the connections. The compatibility is measured when writing the code to simulate the congested clique with the hardware.

- How to simulate the local computation.
- How to simulate the message passing.

Secondly, how the hardware is limiting the performance of the algorithm. Memory, CPU, GPU, network, which one is the bottleneck when the algorithm scales. How the algorithm performs compared to traditional sequential algorithms. By increasing the number of nodes, will it outperform the sequential algorithm with

- Speed comparing to sequential algorithms.
- Number of edges can be processed compared to sequential algorithms.

Scalability:

- Strong scaling and weak scaling (How having more processes speeds up the computation, which part of the hardware will become bottleneck when scaling continuously).
- Time composition for the run when scaling (MPI_time vs local). Does the time complexity hold?

1.1 The MST problem

What is the MST problem

1.2 The algorithm A

See from [1]

1.3 The super-computer Puhti

Used Slurm workload manager [6], to allocate computing nodes. Each node can have processes. The number of total threads is limited, can get 484 * 2 * 20 for the "M" cluster. Total memory available is 192 GiB. Xeon Gold 6230 is the CPU model (ref needed). The nodes are connected with a 100 Gbps HDR100 link, and the topology is a fat tree with a blocking factor of approximately 2:1. [5]

2 CPU implementation

2.1 Congested Clique simulation

A is simulated with OpenMPI and C++. Multiple CPU threads run cooperatively to simulate the congested clique model. Each thread has its dedicated memory space and computing power. Each thread simulates multiple machines in the congested clique model. The local computation of the congested clique is simulated by each thread's local computation while the communication is handled by OpenMPI.

MPI (Message-passing interface) is a message-passing library interface specification that message padding model for parallel computing [7]. Open MPI is an open-source Message-passing Interface implementation.

One CPU thread can hold one or multiple vertices. The thread number is limited. With each thread simulating more than one vertex, larger graphs can be simulated. To simulate more than one vertex, each thread holds all the edges adjacent to the vertices it is responsible for and run the algorithm for each vertex.

When running A, at the beginning, each rank receives the adjacent edges of the node(s) it is responsible for. The algorithm is run iteratively until the MST is found by repeating steps. Each iteration consists of a fixed number of steps and MPI communication.

step 1 → MPI_Alltoall
step 2 → MPI_Alltoall
step 3 → MPI_Scatter
step 4 → MPI_Gather
step 5 and 6 → MPI_Scatter and local computation

To ensure the correctness of the algorithm, the result of A is compared to the result of the prim's algorithm on the same graph.

2.2 Memory limit and graph generation

Each node can have at most 40 threads and has a memory limit of 192 GiB. With n threads for a node, each thread can have at most $192/n$ GiB memory.

The graph is a complete graph represented by a square adj matrix. Each edge is assigned a random weight. There is a naive approach and a distributed approach to generate the graph.

Naive generation: generate the adj matrix on rank 0 and scatter it to all other ranks with MPI_Scatter. Each rank gets a matrix row. Memory eager ($O(n^2)$ memory needed), can't run large enough graph.

Distributed generation: Each rank generates a number of rows of the adj matrix. And share the result with the destination peer with MPI_Alltoall. It saves the memory for graph storage and allows running larger graphs.

3 CPU Algorithm performance

3.1 A time composition

8 processes, 16G memory each. As the number of vertices goes up, local computation becomes the dominant part in the time composition.

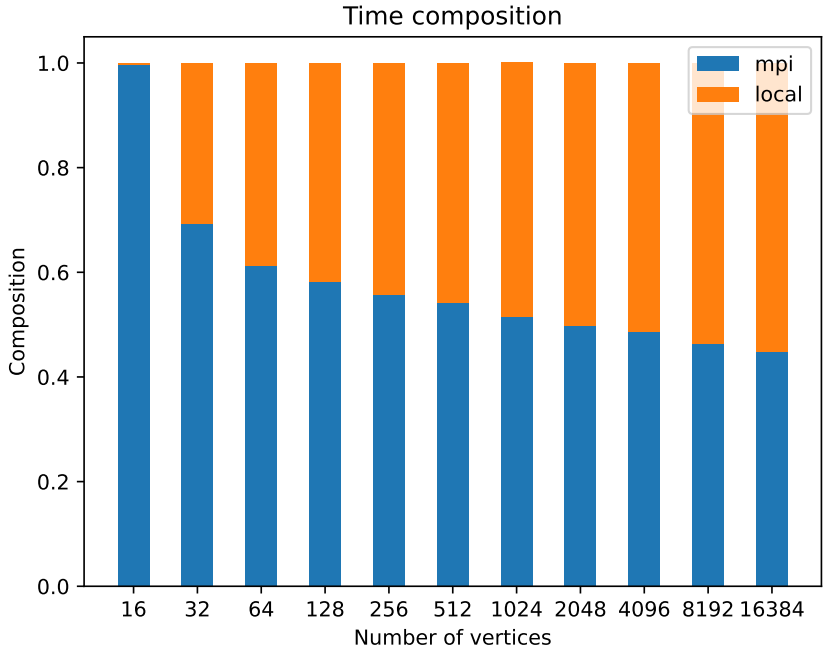

Figure 1: A time composition implemented by MPI

TODO insert time composition by iteration chart.
TODO insert graph with 16 processes
The time decreases as the number of iteration goes up.

3.2 performance compared to sequential algorithm

Prim's algorithm is used as the sequential algorithm to compare the performance of A. It is implemented also in C++ and runs on a single thread on Puhti, with 16G memory.

With 16G memory, prim's algorithm can process 32678 vertices (but not 65356) and $32678 \times 32677/2(5.33 \times 10^8)$ edges using 88 seconds. The bottleneck for prim's algorithm to process more edges is to fit the graph to the memory.

Because of the distributed nature of A, larger graphs can be fit into the memory as the number of threads increases. The program needs more memory for MPI's buffer compared to the prim's algorithms.

3.3 Scalability

3.4 Python vs C++

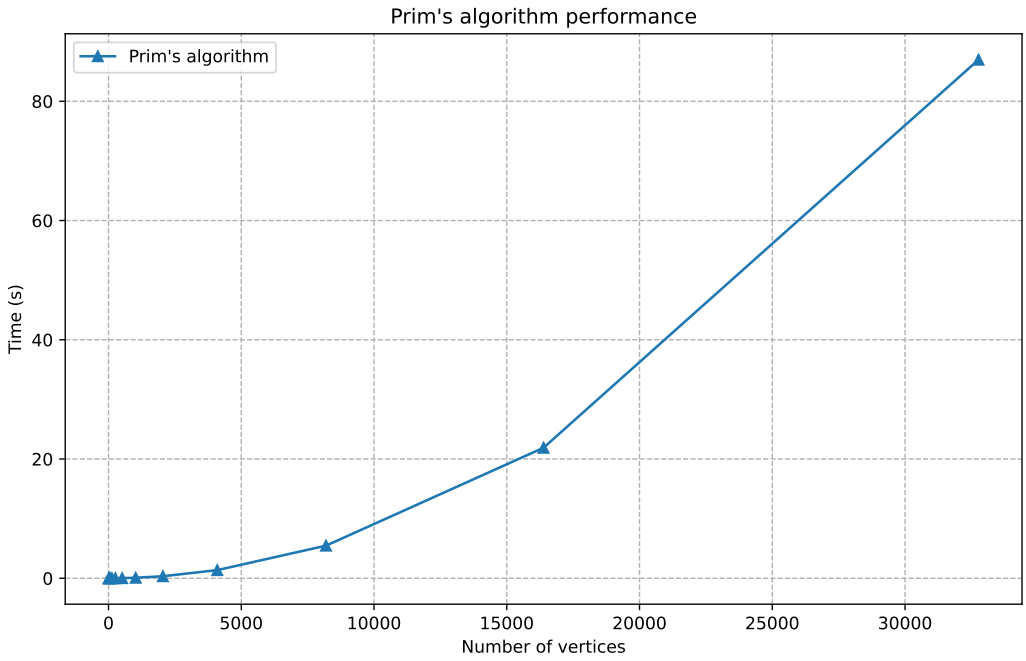

Figure 2: Prim's algorithm performance

4 Cuda implementation

References

This is the list of references to the sources cited in appendix **??**. The list more or less follows the Vancouver style (IEEE). See appendix **??** for a detailed exposition on cross-referencing and bibliography styles. Follow the description there.

[1] Zvi Lotker, Boaz Patt-Shamir, Elan Pavlov, and David Peleg, ”MST Construction in $O(\log \log n)$ Communication Rounds" *SIAM Journal on Computing*, 35(1):120–131, 2005. https://doi.org/10.1137/S0097539704441848

[2] M. Ghaffari, F. Kuhn, and C. Lenzen, "On the power of the congested clique model," *Proceedings of the 2013 ACM Symposium on Principles of Distributed Computing (PODC ’13)*, 2013, pp. 367-376. https://www.researchgate.net/publication/266659337_On_the_power_of_the_congested_clique_model

[3] "Puhti," CSC - IT Center for Science, Accessed: Jul. 16, 2024. [Online]. Available: https://research.csc.fi/-/puhti

[4] "Puhti Service Description," CSC - IT Center for Science, Jun. 2021. [Online]. Available: https://research.csc.fi/documents/48467/640068/Puhti-service description-0621.pdf/e6fd32ae-92ea-2d42-0b6c-9f73c1a4c418?t=1628595451763

[5] "Puhti Computing Systems," CSC - IT Center for Science, Accessed: Jul. 16, 2024. [Online]. Available: https://docs.csc.fi/computing/systems-puhti/

[6] "Slurm Overview," SchedMD, Accessed: Jul. 16, 2024. [Online]. Available: https://slurm.schedmd.com/overview.html

[7] "MPI Forum," Accessed: Jul. 16, 2024. [Online]. Available: https://www.mpi-forum.org/

[8] "Open MPI," Accessed: Jul. 16, 2024. [Online]. Available: https://www.open-mpi.org/